

LadyfingerS Morph Delay

Tiramisù Edition

Computer Music Languages and Systems - Politecnico di Milano

Project documentation

2025–2026

Beretta Pietro
Di Nucci Alessandro
Forlani Serena
Karam Anthony
Messina Giorgio
Neble Manuel
Parlato Domenico

1. System Overview

LadyfingerS Morph Delay is an interactive computer music instrument that combines granular synthesis in SuperCollider with a custom JUCE audio plugin, controlled in real time via a MIDI keyboard (AKAI MPK Mini Mk II). The system implements a rhythm-driven, texture-oriented sound design pipeline where physical gestures (MIDI notes, knobs, joystick) produce complex granular textures and delay effects.

The project integrates three main technological layers:

- SuperCollider: real-time granular synthesis engine, MIDI input handling, OSC routing
- JUCE (LadyfingerS plugin): stereo delay, saturation, pitch shifting, spectral analysis, custom tiramisu-themed UI
- MIDI/OSC communication: bidirectional message routing between hardware, SC, and the plugin

2. Complete Signal & Control Pipeline

Stage	Description
Input	MIDI keyboard (AKAI MPK Mini Mk II) → USB → Windows MIDI
SC: MIDI in	midi_input.scd receives NoteOn/Off, CC, PitchBend via MIDIDef
SC: Mapping	mapping.scd routes events to synth engine, JUCE (OSC), Processing (OSC)
SC: Synthesis	synths.scd: TGrains granular synth, ADSR envelope, voice management
Audio route	SC output → VB-Audio Virtual Cable → JUCE plugin input
JUCE: FX chain	Saturator → StereoDelay (morph/manual) → PitchShifter → output
JUCE: UI	LookAndFeel tiramisù, spectrum analyzer, ENV meter, morph slider
JUCE: OSC in	Receives /cc and /pb messages on port 9001 from SuperCollider
Output	Processed audio → Speaker/Headphone (Realtek)
Visual (opt.)	Processing receives /vmpkNota and /vmpkCC via OSC port 12000

3. SuperCollider Architecture

The SC project is organized as a multi-file system with a single entry point (main.scd) that loads all modules in the correct order.

3.1 File Structure

File	Responsibility
main.scd	Entry point: boots server, loads all modules in sequence
config.scd	Global configuration: OSC addresses, server options, WASAPI device setup
cleanup.scd	Frees all synths and resources before reload
synths.scd	SynthDef definitions, voice management (~playSynth, ~releaseSynth)
mapping.scd	Event handlers: ~handleNoteOn/Off, ~handleCC, ~handlePitchBend
midi_input.scd	MIDIClient init, MIDIDef for NoteOn, NoteOff, CC, Bend
note_state.scd	Note tracking and musical metadata (active notes, polyphony, legato, register classification) — previously underreported in documentation

3.2 Granular Synthesis Engine

The synthesis engine uses TGrains (JUCE-compatible, no juce_dsp required) with a sinusoidally generated buffer as the grain source.

```
SynthDef(\mainSynth, { |out=0, freq=440, amp=0.2, gate=1|
  var sig = Saw.ar(freq);
  var env = EnvGen.kr(Env.adsr(0.01, 0.15, 0.6, 0.6), gate, doneAction:2);
  sig = LPF.ar(sig, (freq*3).min(8000)).tanh.clip(-0.8,0.8);
  Out.ar(out, sig * env * amp ! 2);
}).add;
```

Voice management includes a safety timer (3 seconds max) and a maximum polyphony limit (~maxVoices = 6) to prevent runaway feedback loops on the highest notes.

Each piano key is mapped as a real key triggering an individual voice from a bank of synths. The following subsections detail the full feature set implemented in SuperCollider.

3.2.1 MIDI / Interaction

- Velocity-sensitive note triggering
- NoteOn with velocity 0 treated as NoteOff
- MIDI CC handling
- Pitch bend handling
- Touchpad layer activation using CCs
- Physical pad state tracking to avoid repeated triggering

3.2.2 Note State

- Active note tracking with ~activeNotes
- Polyphony count
- Repeated-note detection
- Legato detection
- Note duration calculation

- Register classification: low / mid / high

3.2.3 Synthesis

The synthesis engine uses an 8-layer synth system, with each layer independently controllable via gain. Synths are dynamically created and released per note, supporting: Sine, Saw, Pulse, FM bell, Filtered noise, Sub oscillator, Glitch/grain-like, and Pad/drone layers.

3.2.4 Safety / Resource Control

- Maximum polyphony limit: $\sim \text{maxVoices} = 6$
- Automatic voice stealing
- Safety release timer if NoteOff is lost
- Cleanup system for MIDIdefs, active synths, and active notes

3.2.5 OSC / External Routing

- OSC note data sent to JUCE
- OSC note data sent to Processing
- CC forwarding to JUCE
- Layer-state messages sent to JUCE and Processing
- Compatibility OSC messages kept for older simple routes

3.3 MIDI Input & Routing

All MIDI input is handled via MIDIdef with four listeners:

- NoteOn/Off: pitch, velocity $\rightarrow \sim \text{handleNoteOn/Off}$, with velocity-0 convention support
- CC: knob number and normalized value $\rightarrow \sim \text{handleCC} \rightarrow$ forwarded as /cc to JUCE on port 9001
- PitchBend: joystick horizontal axis $\rightarrow \sim \text{handlePitchBend} \rightarrow$ mapped to ± 100 cents $\rightarrow$ /pb to JUCE
- All events mirrored to Processing on port 12000 (/vmpkNota, /vmpkCC) for visualization

3.4 OSC Addresses

Address	Port	Description
/cc	9001	SC $\rightarrow$ JUCE: CC number + normalized value [0–1]
/pb	9001	SC $\rightarrow$ JUCE: pitch bend in cents [-100, +100]
/vmpkNota	12000	SC $\rightarrow$ Processing: note state (0/1), MIDI note, velocity
/vmpkCC	12000	SC $\rightarrow$ Processing: CC number + raw value [0–127]

4. JUCE Plugin: LadyfingerS Morph Delay

The JUCE plugin is compiled as a Standalone application and VST3 plugin. It receives audio from SuperCollider via VB-Audio Virtual Cable and processes it through a multi-stage FX chain. All parameters are controllable from the GUI knobs or via OSC messages from SuperCollider.

4.1 Source File Architecture

File	Responsibility
PluginProcessor.h/.cpp	AudioProcessor: OSC receiver, APVTS, processBlock, parameter layout
PluginEditor.h/.cpp	AudioProcessorEditor: GUI layout, paint(), ENV meter, morph bar
TiramisuLookAndFeel.h	Custom LookAndFeel: coffee bean knobs, cream slider, colored buttons
StereoDelay.h/.cpp	Stereo delay with feedback, damping filter, ping-pong mode
DelayModulationEngine.h	Computes delay parameters from morph value using two LFOs
LFOMatrix.h	Multi-waveform LFO (Sine, Triangle, Sample&Hold)
EnvelopeFollower.h	Peak follower with attack/release for envelope-reactive modulation
PitchShifter.h	Dual read-head granular pitch shifter with crossfading
Saturator.h	Soft-clip tanh saturator with drive compensation
SpectrumAnalyzer.h	Real-time FFT spectrum display using juce_dsp

4.2 Audio FX Chain

Audio flows through the following stages in processBlock():

- Envelope Follower: tracks signal amplitude, stored as atomic float for GUI display
- Morph Engine: if MORPHON is active, DelayModulationEngine interpolates all delay parameters from a single MORPH value (0=ambient, 1=glitch). Otherwise, parameters are read directly from APVTS.
- Saturator: tanh soft-clipping with drive compensation (1x–20x)
- StereoDelay: stereo delay with damping filter, ping-pong, and configurable feedback
- PitchShifter: dual read-head granular pitch shifter controlled by PITCH (coarse, ± 24 semitones) and PITCHFINE (joystick, ± 100 cents)
- SpectrumAnalyzer feed: processed buffer sent to editor for real-time display

4.3 Delay Modulation Engine

The DelayModulationEngine implements the core creative concept: a single MORPH parameter continuously interpolates between two extreme sound characters.

Parameter	Morph=0 (Ambient)	Morph=1 (Glitch)	LFO Modulation
Delay Time	700 ms	25 ms	Sine LFO @ 0.2 Hz
Feedback	0.30	0.93 + ENV boost	Env follower boost
Damping Hz	1200 Hz	14000 Hz	Triangle LFO @ 0.4 Hz
Ping-Pong	OFF	ON (morph > 0.45)	Threshold-based

4.4 Pitch Shifter

PitchShifter.h implements a dual read-head granular pitch shifter without requiring juce_dsp. Two read heads traverse a circular buffer at a rate determined by the pitch ratio, with sinusoidal crossfading to avoid artifacts at the reset point.

- Coarse pitch: controlled by knob 8 (CC 8), range ± 24 semitones
- Fine pitch (microtonality): controlled by joystick horizontal axis, range ± 100 cents. Note: this feature is not implemented in SuperCollider, as SC does not read the joystick horizontal axis; the pitch bend message is passed through to JUCE directly via OSC.
- Buffer size: 8192 samples (~170ms at 48kHz)
- Crossfade window: sinusoidal, based on distance between read head and write head

4.5 APVTS Parameter Map

ID	Range	CC Mapping	Description
MORPH	0.0 – 1.0	CC 1	Ambient ↔ Glitch interpolation
WET	0.0 – 1.0	CC 2	Wet signal level
DRY	0.0 – 1.0	CC 3	Dry signal level
FEEDBACK	0.0 – 0.95	CC 4	Delay feedback amount
DELAYTIME	1 – 2000 ms	CC 5	Delay time (log scale)
DAMPING	500 – 20000 Hz	CC 6	Damping filter cutoff
DRIVE	1.0 – 20.0	CC 7	Saturator drive amount
PITCH	0.0 – 1.0	CC 8	Pitch coarse (± 24 semitones, center=0)
PITCHFINE	0.0 – 1.0	Joystick /pb	Pitch fine (± 100 cents, microtonality)
MORPHON	bool	—	Enable/disable morph engine
DRIVEON	bool	—	Enable/disable saturator
PINGPONG	bool	—	Ping-pong delay mode



5. User Interface — Tiramisù Edition

The plugin UI is fully custom-designed using a `TiramisuLookAndFeel` class that overrides JUCE's default rendering for all components. The visual theme references the photo of the "Ladyfingers TiraSynth-8" — a tiramisu-shaped synthesizer — as its aesthetic inspiration.

5.1 TiramisuLookAndFeel

The custom `LookAndFeel` overrides three key rendering methods:

drawRotarySlider — Coffee Bean Knob

Each rotary knob is drawn as a coffee bean: an ellipse with a central curved groove (drawn as a cubic bezier path), a highlight ellipse, and a pointer dot. The entire bean rotates with the slider value. The pitch knob uses a `PitchBeanLookAndFeel` subclass that renders the bean in mascarpone cream instead of coffee brown.

drawLinearSlider — Morph Slider

The horizontal morph slider uses a dark espresso track with a cream fill, and a thumb shaped like a small coffee bean with groove.

drawToggleButton — Colored Indicator Buttons

Toggle buttons are rendered as colored filled circles: blue (Ping Pong), red (Morph ON), green (Drive ON), matching the aesthetic of colored jack connectors on hardware synthesizers.

5.2 Background Decorations

- Coffee bean shapes: drawn at low opacity (12%) in free areas of the background — not overlapping controls
- Cream piping border: series of overlapping oval blobs along the panel perimeter, simulating piped pastry cream with highlight and shadow
- Cream swirls: layered ellipses in the top-right corner suggesting whipped cream decoration
- Dark espresso panel with subtle gradient from coffee to chocolate

5.3 Real-time Visual Feedback

- ENV Meter: vertical bar on the right side, reads `envelopeValue` atomic float from processor. Color transitions from roast brown (quiet) to mascarpone cream (loud)
- Morph Bar: horizontal progress bar below the title showing current morph position with a cream thumb indicator

- Spectrum Analyzer: real-time FFT display using `juce_dsp` (`juce::dsp::FFT + WindowingFunction::hann`), 2048-point analysis, 128-bin display with logarithmic frequency axis

6. Interaction Design

The system offers two interaction layers that can be combined during performance:

6.1 Physical Interaction (MIDI Keyboard)

- Notes (NoteOn/Off): play melodies or rhythmic patterns → SC granular synthesis
- Knobs 1–7 (CC 1–7): control MORPH, WET, DRY, FEEDBACK, TIME, DAMPING, DRIVE in JUCE
- Knob 8 (CC 8): pitch coarse — shift the processed delay output up/down by semitones
- Joystick horizontal (Pitch Bend): microtonality — fine-tune the pitch continuously between semitones (± 100 cents)

6.2 GUI Interaction

- All sliders and knobs respond to mouse drag and scroll
- Toggle buttons activate Ping-Pong, Morph Engine, and Saturator modes
- Morph slider provides a single-gesture control that simultaneously transforms delay time, feedback, damping, wet/dry, and ping-pong

6.3 Novel Mapping: Microtonality via Joystick

The joystick's horizontal axis (pitch bend) is mapped to ± 100 cents in JUCE's PitchShifter. This allows the performer to play a melody on the keys and simultaneously push the joystick sideways to continuously detune the processed output between semitones — enabling microtonal expression on a standard MIDI keyboard without re-tuning. Note: fine pitch control is currently not implemented at the SuperCollider level, as SC does not read the horizontal axis of the joystick; the pitch bend value is forwarded directly to JUCE via OSC.

6.4 Processing Visualization: Game of Life

In Processing, the system implements a musical reinterpretation of Conway's Game of Life — a classic computational simulation of emergence. The world does not start from a random state; instead, MIDI notes generate new cells, coloured according to their pitch.

6.4.1 Original Rules

Each cell updates its state according to the state of the 9 cells surrounding it:

- Any live cell with fewer than two adjacent live cells dies, as if by isolation.
- Any live cell with two or three adjacent live cells survives to the next generation.
- Any live cell with more than three adjacent live cells dies, as if by overcrowding.
- Any dead cell with exactly three adjacent live cells becomes a live cell, as if by reproduction.

6.4.2 Plugin Effect Modifications

Some plugin effects modify the original rules of the game:

- **Delay** changes the speed of the simulation: increasing its value slows down the generation updates, while decreasing it makes the world evolve more quickly.
- **Feedback** increases the fertility of the system: in addition to the standard birth condition of 3 adjacent live cells, it introduces a probability of birth even with 2 or 4 adjacent live cells.
- **Damping** makes the world more hostile: although live cells with 2 adjacent live cells would normally survive, increasing damping raises their probability of dying.
- **Drive** increases the chaos of the system by making cell death and survival probabilistic.

7. Setup & Startup Guide

7.1 Required Software

- SuperCollider 3.14+
- JUCE 8.0.12 (for building from source)
- VB-Audio Virtual Cable (free, vb-audio.com)
- Visual Studio 2026 Community with C++ Desktop workload
- loopMIDI (optional, for MIDI splitting)

7.2 Audio Device Configuration

SuperCollider must be configured to output to VB-Audio Cable Input, so JUCE can receive it on Cable Output:

```
Server.default.options.outDevice = "Windows WASAPI : CABLE Input (VB-Audio  
Virtual Cable)";  
Server.default.options.inDevice  = "Windows WASAPI : Microphone Array  
(Realtek(R) Audio)";  
Server.default.options.sampleRate = 48000;
```

```
s.boot;
```

In the JUCE Standalone plugin, set:

- Audio Input: CABLE Output (VB-Audio Virtual Cable)
- Audio Output: Speaker / Headphone (Realtek)
- Sample Rate: 48000 Hz

7.3 Startup Order

Components must be started in this exact order to avoid MIDI device conflicts:

- 1. Connect AKAI MPK Mini Mk II via USB
- 2. Start VB-Audio Virtual Cable (should autostart)
- 3. Open SuperCollider → run main.scd
- 4. Verify post window: 'PROJECT BACKBONE READY'
- 5. Open LadyfingerS Standalone plugin
- 6. Set audio I/O in plugin Options

7.4 Building from Source

```
cd C:\CMLS_PROJECT\TESTING
cmake -B build -S . -G "Visual Studio 18 2026" -A x64
cmake --build build --config Release
```

The VST3 plugin will be at:

```
build\TESTING_artefacts\Release\VST3\TESTING.vst3
```

8. Novelty and Creative Contribution

The project contributes to the field of computer music interaction in several ways:

- Morph paradigm: a single gesture (one slider or one knob) transforms an entire multi-dimensional parameter space, reducing cognitive load during performance while maintaining expressive depth
- Microtonality on standard hardware: the joystick pitch bend → PitchShifter mapping enables microtonal performance without specialized hardware or retuning
- SC-JUCE live integration: SuperCollider handles synthesis and MIDI routing; JUCE handles DSP effects and visualization. Both communicate in real time via OSC, forming a heterogeneous but tightly coupled system
- Envelope-reactive feedback: the delay feedback is automatically boosted by the envelope follower, making the system respond dynamically to how hard the performer plays
- Custom aesthetic: TiramisuLookAndFeel with coffee bean knobs, cream piping border, and spectrum analyzer creates a distinctive visual identity that reinforces the playful character of the instrument

9. Known Limitations

- MIDI device exclusivity on Windows: SC and JUCE cannot both open the MPK simultaneously. SC must start first; JUCE receives CC data via OSC instead of direct MIDI
- Occasional note loop on highest MIDI note: partially mitigated by safety timer (3s) and polyphony limit, but may persist under very fast repeated triggers
- Pitch shifter artifacts: the dual read-head approach produces audible artifacts at high pitch ratios ($>\pm 12$ semitones) and on transient-rich material
- PitchShifter CPU cost: per-sample circular buffer processing with two read heads adds measurable CPU load at low buffer sizes